



AI原生研发团队进化指南

从敏捷(Agile)到智能(AI-Native)的实战跃迁

Ethan Jiang

敏捷跑了这么久，速度为何还是上不去？

"眼看高楼平地起，轮到自己却还在一块砖一块砖地垒。"

你是不是也这样？团队里 Jira 用的炉火纯青，Sprint 排得满满当当，每日站会开得风生水起，但作为研发管理者和PM，心里那股焦虑劲儿却挥之不去：

✅ 感觉沟通是顺畅多了

看板上流程清晰，需求流转看得见，大家协作效率肉眼可见。

✅ 跑偏的风险也大大降低

短周期迭代，发现不对劲立马就能调整，小步快跑避免了大坑。

❌ 但最核心的交付速度，就是没有真正飞起来

因为真正写一行代码、改一个Bug、测一个功能，所需的时间，一秒钟都没有少。活儿还是得人一点点做出来。

📌 💡 **Takeaway：** 敏捷解决了协作问题，但从未解决执行速度问题。真正的瓶颈，一直藏在最底层。

致命误区：把 AI 当作提效的插件

"给马车装上喷气引擎，流程的骨架必定会散架。"

企业在拥抱 AI 时最常见的错误做法：

- 仅仅买一堆 Copilot 工具，把它当成程序员个人的"代码补全助手"。
- 依然按照传统敏捷按"人天/故事点"排期。
- 产品的交接物依然是一张画得非常唯美的高保真原型图。

❏ **🔥 结局必然是失控的：** 如果我们仍然按传统敏捷的思路，AI 极快的代码生成速度只会导致测试压力剧增、技术债成倍积累。

破局点：AI 具备了极其强大的端到端执行力

"我们现在正在用英语编程。这太疯狂了。提示词就是程序，写提示就是在写功能。"

—— Andrej Karpathy，前OpenAI创始成员 / 前特斯拉AI总监，2025年YC AI Startup School演讲

 2025年YC冬季批：1/4的初创公司已让AI自动生成95%以上的代码。软件工程的奇点，已经到来。

过去：人是执行的最小单元

5个角色串行——产品写PRD、设计出图、后端写接口、前端联调、QA手工测试。最快也要数天，每行代码都要人敲。

现在：AI 可以一口气吃掉整个执行层

给一份清晰的Spec，AI分钟级内并发生成后端接口、前端页面、单元测试、接口文档。不是辅助——是替代整个执行流水线。

AI 能做到什么程度？

完整CRUD生成；前后端联调代码自动生成；测试用例自动生成并执行；Bug自动定位、修复、提PR。

那瓶颈在哪里？

不在执行。在定义——你能不能给AI一份边界清晰的Spec？这是新时代唯一的核心竞争力。

 **Takeaway：** 真正卡住交付周期的，不再是缺乏熟练的开发人员——而是缺乏能驾驭 AI 的定义者。

从敏捷到 AI-Native：四步跃迁



阶段1：诊断现状

- 敏捷的天花板在哪里？
- 为什么执行速度始终卡住？
- 致命误区：把 AI 当插件



阶段2：认知重塑

- 围绕 AI 重新设计组织
- PO / 架构师 / Dev / QA 的角色进化
- 新的人机协作语言：Spec



阶段3：方法落地

- 为什么传统 PRD 失效？
- Spec 编程 × 三层基石
- 并发飞轮 × 三明治生产流



阶段4：稳健跃迁

- Pragmatic 落地路线图
- 存量老项目的剥洋葱战术
- 从这里出发：现在就能做的三件事

重塑认知底座：围绕 AI 重新设计组织

"把 AI 当作全能的主力员工，让人去打配合。"

组织切分逻辑按职能切分（前端/后端/测试/产品）按"定义能力"切分（谁能给AI下清晰指令，谁是核心资产）人的核心价值写代码、画原型、手工测试——执行本身就是价值定义规则、约束边界、验收质量——执行已外包给AIAI的定位辅助工具，帮人提速10%~30%全栈执行主力，人负责定义和验收最大瓶颈执行人力不足、排期太满定义质量不足——Spec模糊，AI就跑偏

📌  **Takeaway:** 这不是一次工具升级，而是一次组织操作系统的重装。旧的假设——"人是最小执行单元"——已经失效。新的假设是：AI是执行主力，人是定义者与守门人。

新的组织形态：你管的不再只是人

"当数字员工（Agent）正式入编，管理者面对的不再是一支纯人类团队——而是人与 Agent 共同构成的混合组织。"

这不是工具升级，而是组织形态的根本变化。传统管理学建立在"人是最小执行单元"的假设上。这个假设，正在被打破。

	传统团队	人 + Agent 混合团队
管理对象	管理人的状态、情绪、产出	同时管理人的判断力 + Agent 的任务分配与质量验收
编制逻辑	按职能划分：前端/后端/测试/产品	按"定义能力"划分：谁能给 Agent 下清晰的指令，谁就是核心资产
绩效评估	评估个人代码产出、需求完成率	评估人驾驭 Agent 的效率：Spec 质量、验收准确率、技术债控制
协作节奏	人等人，串行排队	人定义，Agent 并发执行，人验收

📌 **Takeaway：** AI 原生转型的本质，是一场组织重构——不是裁员，不是换工具，而是重新定义"谁做什么"、"怎么协作"、"用什么衡量价值"。

全员 AI：没有例外，没有旁观者

"在AI原生团队里，不用AI的人，不是落后者——而是堵路者。"

AI原生团队的协作节奏已发生根本性改变——工作交付物是Spec，沟通载体是结构化文档，执行主力是Agent。如果团队中有人不使用AI，不懂AI的语言，他将成为整个工作流的瓶颈，阻碍所有人。这不是个人选择，而是组织协作的基础设施问题。

协作语言已经变了

团队的交接物从口头确认、Jira评论变成了结构化Spec。不会读写Spec的人，无法参与协作闭环。

节奏已经变了

AI并发执行的速度远超人工串行。一个不用AI的环节，会让整个并发飞轮强制降速，退回串行等待。

评估标准已经变了

个人价值不再由代码行数衡量，而是由驾驭AI的质量决定。拒绝使用AI，等于主动放弃自己的核心竞争力。

📌 **⚠️ 管理者的责任：**全员AI化不是口号，而是需要配套的培训、工具、流程和考核机制。推动它落地，是管理者在AI原生转型中最重要的组织动作之一。

Product Owner：从原型描绘者，到逻辑约束的定義者

"你画了无数唯美的高保真原型，最终在 AI 秒级生成中失去意义。"

传统产品经理	AI原生产品专家
提供几百页带完整 UI 交互逻辑的冗余 PRD	提供极其干练的结构化 Spec (规范约束)
偏重考核高保真原型的精细度和交互美感	极度优先考察对"角色旅程、边缘异常用例"的穷尽洞察力

📌 ⚖️ PO 的核心价值向两端转移： 向上做战略判断，向下定义极其严密无歧义的验收标准。把人听得懂的需求，翻译成机器能直接执行的逻辑约束。

架构师 / Scrum Master：从任务分配者，到 AI 脚手架的缔造者

AI 写代码太快，靠人力事后 Review 的防线，必然崩溃。

过去的职责	AI原生的新职责
核心产出物：Story Point估算表、任务分派看板、手工Code Review意见	核心产出物：项目级AI准则文档（.cursorrules / System Prompt脚手架）、技术契约（OpenAPI）、防幻觉约束规则
对AI的态度：AI是辅助工具，出了问题人工兜底修改	对AI的态度：AI是执行主力，出了问题回溯Spec定义层，而非手动修补产出
最高价值动作：协调人与人之间的沟通和排期	最高价值动作：设计让AI不跑偏的系统约束；构建极度健壮的上下文工程脚手架
衡量标准：Sprint速度、故事点完成率	衡量标准：AI产出的一次性通过率、技术债增长率、Spec清晰度评分

📌 建立极度健壮的准入机制，让 AI 在防幻觉脚手架内狂奔，而不是任由它制造技术债。

开发者 Dev：从搬砖执行者，到深架构的质量守门人

能被 AI 轻易生成的执行能力，再也无法构成开发者的护城河。

过去的职责	AI原生的新职责
前后端分离，各自排期写重复度极高的业务CRUD代码	驾驭AI端到端完成需求落地；不再区分前后端，一人全栈杠杆
熟练掌握框架语法、记住API用法	架构判断力：能识别AI产出是否引入技术债、安全漏洞、性能隐患
逐行审查业务逻辑，耗时耗力	聚焦架构层审查：AI生成的代码结构是否符合系统约束，边界是否清晰
代码产出量、熟悉某个框架	对系统全局的深度理解；能给AI下达精准指令并准确验收产出质量

📌 🏆 黄金价值：从写代码的执行者，进化为判断 AI 产出质量的审阅者与把关人。

QA：从流水线末端的点点点，到 AI 时代的质量防线缔造者

没有自动化防护网兜底，AI 跑得越快，系统死得越惨。

过去的职责	AI原生的新职责
处于流水线最末端，高度依赖手工黑盒点点点	成为测试策略的最强定义者；用自动化测试体系构筑AI无法逾越的质量防线
手工测试用例，靠人力跟上开发节奏（跟不上）	命令AI批量生成单元测试、集成测试、端到端测试；人负责定义覆盖策略
发现Bug → 提Jira → 等开发修复 → 回归验证	发现Bug → 回溯Spec定义层 → 驱动AI修复并自动回归验证
保证上线前没有明显缺陷	设计并维护整个自动化防护网；AI跑得越快，防线越重要——QA地位空前拔高

  QA 是保障 AI 原生团队不翻车的绝对底线。地位不是被削弱，而是空前拔高。

角色变了，协作方式也必须跟着变

PO、架构师、Dev、QA——每个人都从过去的“执行者”变成了“定义者”和“守门人”。但一个新的问题也随之浮现：

当每个人都在定义规则、约束和标准，团队用什么语言来对齐？用什么载体来让 AI 无歧义地执行？

PO / 产品专家

从原型描绘者 → 逻辑约束的定义者。
核心武器：结构化Spec与验收标准。

架构师 / SM

从任务分配者 → AI脚手架的缔造者。
核心武器：技术契约与上下文工程。

开发者 Dev

从搬砖执行者 → 深架构的质量守门人。
核心武器：架构判断力与代码审查。

QA

从末端点点点 → 质量防线的缔造者。
核心武器：自动化测试体系与防护网。

值得注意的是，这四个角色并不对应四个人。AI大幅压缩了执行成本，一人同时扮演多个角色——小团队、多角色、一人多帽——在AI原生团队中不仅可行，更是常态。

❏ 四个角色，一个共同方向：从执行者转型为定义者。旧的交接方式——冗长PRD、口头确认、Jira评论——在AI面前已彻底失效。团队需要一套新的共同语言：Spec。

一切执行的基础：为什么传统 PRD 失效了？

"人类的'脑补'是阻断 AI 自动化的剧毒。"

为什么给 AI 喂传统 PRD，出来的总是奇形怪状的废品？

模糊性

人类写文档充满了天然歧义和跳跃，传统敏捷靠会议和高频沟通来"填补这些空白信息"。

碎片化

零散的跟帖记录、口头确认，AI 永远无法准确抓取。

不可执行性

传统 PRD 描述的是"期望结果"，而非"执行规则"。AI 需要的是边界条件和验收标准，不是产品愿景。

📌 **认知鸿沟：**让 AI 去"猜"人类语境下的默契，是团队产生最大挫败感的根源。

新核心抓手：Spec 编程 (Spec as Code)

"用规范和契约作为团队沟通的首要公民。"

传统团队接纳 AI 最高效的切入点，叫做 Spec 编程——核心思想是：把规范（Spec）当作代码一样严格对待，让它成为人与 AI 之间唯一的沟通媒介。

Spec即指令

团队沟通传递交接的中心不再是 Jira 任务详情，而是高度结构化的 Spec（规范与契约）。

消除歧义

将需求直接转化为极其严谨、机器立刻能读懂的边界条件约束。

  **Takeaway:** 学会写高质量的 Spec，是你向 AI 下达确定性执行军令状的唯一方式。

驱动 AI 的三层 Spec 基石

在动手干活前，请先准备好弹药库：

Spec层级	载体形态	解决什么问题
产品约束 Spec	高度结构化的 Markdown (User Journey / Acceptance Criteria)	给 AI 定义：业务角色、流程流转闭环、核心验证标准
技术契约 Spec	Swagger / OpenAPI 接口文档	给 AI 定义：系统交互的心智基石，这是 AI 写后端的指南针
规则防坑 Spec	.cursorrules / System Prompt 脚手架	给 AI 定义：团队编码规范、UI 组件库限制、绝不允许踩的技术坑

  **地基重塑：** 任何缺乏这三层基石直接跑去让 AI 敲代码的行为，都是在盲人摸象。

Spec 有了，但飞轮还没转起来

Spec 是静态的弹药，但 AI 执行是动态的。同样一份 Spec，不同的编排方式——任务拆解的粒度、上下文传递的顺序、并发与串行的节点选择——最终得到的结果可能天差地别。

Spec 是弹药

定义清晰、边界严密的 Spec 是前提。没有它，AI 无从瞄准。

编排是扣扳机的方式

如何拆解任务、如何传递上下文、何时并发何时串行——编排决定 AI 能否真正打中靶心。

📌 **Takeaway:** Spec 解决"说清楚"，编排解决"跑起来"。两者缺一，飞轮都转不动。

重塑 Sprint：从串行流水线，到并发飞轮

传统的线性铺排在 AI 面前轰然倒塌。

传统的流水线：定排期 → 画原型 → 写后端接口 → 前端联调 → 测试走查。每一步都在等上一步的"人"完工。

传统 Sprint 节奏	AI原生 Sprint 节奏
按人天/故事点估算工时	按 Spec 完成度衡量就绪状态
前端、后端、测试顺序排队	Spec 锁定后，AI 并发生成前后端+测试用例
人是执行主力，AI 是辅助	AI 是执行主力，人是定义者与验收者
Sprint Review 靠人工演示	Sprint Review 靠 CI/CD 自动化报告 + 人工体验验收

📌 ⚡ **Takeaway：** Sprint 的价值不再是"这两周写了多少代码"，而是"定义了多少清晰的 Spec，并让 AI 完成了多少高质量的并发执行"。

新一代生产流： "定义 - 生成 - 验证" 三明治模型

忘掉前后端分工，迎来全新的人机大协同。

新生产流阶段	敏捷对应环节	核心主导	动作目标
定义层	Sprint Planning	 人类主导	打磨极简而严密的 Spec 。这部分一旦定义清楚无歧义，工作其实已经完成了大半。
生成层	In-Sprint Execution	 AI主导	人放手，AI瞬间并发吞入所有 Spec，多线程自动提 PR、写测试例、出文档。
验证层	Sprint Review	 人机协同	依靠极度刚性的 CI/CD 流水线刚性拦截，辅以人类做最终"体验验收"。

  **Takeaway：** 把 80% 的人类集体心智前置砸在"定义层"——规则定义到极致，AI 的执行才能无懈可击。

新模型很美，但现实是：你的团队背着历史包袱

“定义-生成-验证”三明治模型在新项目上跑起来很顺，但大多数企业面对的现实是——几十万行的老代码、没有测试覆盖的遗留系统、习惯了旧流程的团队。如果不考虑这些现实约束，直接把新模型套上去，结果只会是一场灾难。

1

第一层：代码地雷阵

老代码没有测试覆盖，AI 一改就可能引爆未知的连锁反应。这是最底层的物理障碍。

2

第二层：认知惯性墙

从“人工执行”切换到“AI主导生成”，需要认知和工作方式的双重转变。地雷没排完，人还没转变，新模型根本落不了地。

3

第三层：组织风险底线

核心业务不能停。前两层没处理好就激进切换，是在赌公司的命。

所以，AI原生转型不是一场革命，而是一场有纪律的演进。

📌 🚫 **Takeaway:** 转型路径的优先级：先建自动化防线（CI/CD + 测试覆盖），再在防线内逐步放开 AI 的执行权限——而不是反过来。

Pragmatic Adoption: 稳妥的落地路线图

对于背着沉重历史技术债的企业，一刀切全是灾难。

向 AI 原生跨越的四大铁律：

1

绝不停转核心业务

所有 AI 试点必须是增量叠加，不是替换。

2

不执念大换血

核心班底的业务判断力，是 AI 永远无法替代的资产。

3

克制技术狂热

千万行老代码不是 AI 的食材，是地雷阵。

4

先筑护城河

DevOps 流水线和 CI/CD，必须是第一发子弹。

📋 🐢 **Takeaway:** 别看跑得多快，要看刹车灵不灵。AI原生落地，稳健优先于激进。

存量老项目(Legacy)的剥洋葱战术

一步错满盘输：切忌让 AI 上来就去乱改核心老旧逻辑。

推荐的破局三部曲：

01	02	03
第一步（兜底防线）	第二步（试水温）	第三步（下深水）
圈定边界，命令 AI 给老代码疯狂补全缺失的单元测试和集成测试。（人在定边界，AI在织网）	指挥 AI 在隔离的分支中，处理那些边缘的 Bug 修复与非核心迭代。（低风险摩擦）	当自动化测试防线极其坚固时，放出 AI，多线程自动化地去重构原本因为太昂贵而始终下不去手的历史技术债。

📄 🍷 **重构红利：** 能够极其低廉、快速地清理历史债务，是传统企业使用 AI 获取的最大隐性红利。

学会驾驭 AI：先摸清它的边界

AI 解决问题的方式和人有共性，但差异更大——它的速度不一样，它的习惯不一样，它听得懂的语言也不一样。

探索这条边界，没有捷径。需要通过大量的实际尝试，在反复校准中感知 AI 的能力边界——它在哪里游刃有余，在哪里开始失准。

边界会随模型升级而漂移

今天积累的经验，在新模型面前可能需要重新校准。这意味着"持续校准 AI 边界"应当成为团队的长期能力建设，而非一次性的学习任务。

AI 笨拙时，先反思自己

当 AI 的表现不及预期，问题往往不在模型本身，而在于输入信息的质量——上下文不足、边界模糊、指令存在歧义。模型能力越强，这一规律越为显著。

  **Takeaway：** 驾驭 AI 的核心能力，不是会用工具，而是会提问、会定义、会反思——这是一种需要刻意练习的新型素养。

最重要的纪律：AI 的产出物，人不许改

"人只做一件事：驾驭 AI，让 AI 解决问题。AI 的产出物——无论是代码还是文档——人不得手动修改。"

这是 AI 原生团队最难内化、也最核心的一条工作纪律。它不是技术层面的约束，而是对工作方式的根本性重塑。

为什么不许改？

一旦人工介入修改 AI 的产出物，团队便重新滑回'人是执行者'的旧模式。后续 AI 重新生成时，手动修改将被覆盖，或形成难以追溯的版本混乱。

出现问题，回到上游

应回溯至定义层：审视 Spec 是否足够清晰、指令是否存在歧义、上下文是否完整，再驱动 AI 重新生成。问题的根源在定义层，而非产出层。

这条纪律的真正价值

它强迫整个团队把心智投入到"定义"而不是"修补"。当你无法靠手动兜底，你才会真正学会如何驾驭 AI。

📌  记住这条纪律的本质：它不是在限制你，而是在强迫你把心智投入到真正有价值的地方——定义，而非修补。

从这里出发：你现在就能做的三件事

"以前，团队比拼的是：'谁的敏捷体系能更顺畅地驱动人去敲出代码'。而现在，核心竞争力变成了：'你的组织有没有能力，建立起一套坚不可摧的上下文与防线体系，极度清晰地告诉全能的 AI，我们到底要建造一个什么东西。'"

现在就能做的三件事

本周：评估AI就绪度

召集 PO + 架构师 + QA，对齐"我们现在的 Spec 质量够不够让 AI 直接执行"。

本月：小范围试点

选一个低风险的新需求，用"定义-生成-验证"三明治模型跑一次完整试点。

本季：强化自动化防线

把 CI/CD 自动化覆盖率作为团队 OKR 的硬指标，而不是软目标。

📌 **最后一个问题留给你：** 你的团队，今天能写出一份让 AI 直接执行的 Spec 吗？如果还不能——这就是你最值得投入的第一件事。